AI для .NET-розробника

Том 5 · Практика .NET

Розділи 13–15

Розділ 13. Тести, рефакторинг, міграції, документація

13.1. Юніт-тести з AI

Одна з найкращих задач для AI. Але **якість тесту = що саме він перевіряє**.

```
Напиши xUnit-тести для OrderCalculator (#file:OrderCalculator.cs).
- Використовуй FluentAssertions; назви Метод_Сценарій_Результат.
- Покрий: порожнє замовлення, знижка 100%, від'ємна кількість (очікуй
ArgumentException),
  округлення до 2 знаків (MidpointRounding.AwayFromZero).
- Не мокай OrderCalculator. Після створення виконай dotnet test.
```

Перевірка «чесності» тесту

1. Навмисно зламайте логіку — тест має впасти (poor man's mutation testing).
2. Перевірте, що Assert перевіряє результат, а не просто «не впало».
3. Переконайтеся, що тест не копіює реалізацію (інакше він завжди проходить).
4. Для покриття використовуйте `dotnet test --collect:"XPlat Code Coverage"` та інструменти на кшталт Stryker.NET.

TDD з AI

Спершу попросіть написати тести за специфікацією (вони мають падати), закомітьте, потім просіть реалізацію, що проходить тести. Так ви контролюєте вимоги, а не агент.

13.2. Рефакторинг

- **Спочатку тести** (характеризаційні), потім зміна структури.
- Малі кроки: «витягни метод», «заміни switch на стратегію», «переведи на record».
- Просіть зберігати поведінку й публічний API.
- Roslyn-аналізатори й `dotnet format` — детермінований помічник, не залежний від моделі.

Задача	Формулювання
Nullable	«Увімкни nullable у проєкті та виправ попередження без підавляння (!)»
Async	«Знайди синхронні I/O виклики в цьому шарі та переведи на async»
DI	«Прибери new SmtpClient() — введи інтерфейс і впровадь через DI»
Продуктивність	«Знайди N+1 у цьому EF-запиті; запропонуй Include/проєкцію і поясни SQL»

13.3. Міграції та оновлення версій

- Оновлення .NET: попросіть агента скласти **план** (TargetFramework, пакети, breaking changes), потім виконувати по проєктах із `dotnet build/test` на кожному кроці.
- Перехід на Minimal API, з Newtonsoft на System.Text.Json, з EF6 на EF Core — найкраще ділити на малі PR.
- Для офіційних інструментів міграції (Upgrade Assistant / GitHub Copilot app modernization — перевіряйте актуальну назву й доступність) агент може допомагати, але рішення залишаються за вами.
- Не довіряйте «пам'яті» моделі про breaking changes — додайте в контекст офіційні release notes або Learn MCP.

13.4. Документація

- **XML-коментарі:** «Додай XML-документацію до публічних членів, без очевидних повторів назви».
- **README:** агент із доступом до репозиторію добре складає розділи «Запуск», «Архітектура».
- **ADR** (Architecture Decision Records): обговорюйте варіанти з AI, фіксуйте рішення.
- **Повідомлення комітів / описи PR:** Copilot генерує чернетку, ви редагуєте.
- **Діаграми:** Mermaid-код у Markdown (GitHub його відображає).

````
```mermaid
flowchart LR
  Client --> API --> Service --> EFCore --> DB[(SQLite)]
```
````

13.5. Налагодження

1. Дайте повне повідомлення, стек-трейс, відтворення.
2. Попросіть **гіпотези** та способи їх перевірки, а не одразу правку.
3. Використовуйте дебагер Visual Studio; агент може допомогти читати змінні й журнали.
4. Після виправлення попросіть написати регресійний тест.

Завдання 13. Візьміть метод без тестів у вашому проєкті. (1) Згенеруйте тести. (2) Зламайте метод — переконайтеся, що тести падають. (3) Зробіть рефакторинг із AI. (4) Додайте XML-документацію.

Розділ 14. Рев'ю, безпека, ліцензії, приватність

14.1. AI як рев'юер

- Copilot code review на PR на GitHub і в IDE; власний агент-рев'юер (розділ 9).
- Використовуйте як **першу лінію**: ловить дрібниці, звільняючи людину для архітектурних питань.
- Не замінює людське рев'ю й не гарантує повноти — можливі помилкові й пропущені зауваження.

Чекліст рев'ю коду, згенерованого AI

Область	Питання
Коректність	Чи робить те, що просили? Крайні випадки?
Існування API	Чи всі методи та пакети реальні й актуальні?
Async	Немає `.Result` / `.Wait()`, async void, втрачених токенів?
Ресурси	Dispose/using, HttpClient через IHttpClientFactory?
Безпека	Валідація входу, авторизація, секрети, SQL?
Тести	Є, змістовні, детерміновані?
Простота	Немає зайвих абстракцій і «над-інженерії»?

14.2. Безпека коду

Типові вразливості, які може привнести AI-код (орієнтуйтесь на OWASP Top 10):

- **SQL-ін'єкція:** конкатенація рядків у запитах → параметризація/EF.
- **Відсутня авторизація** на ендпоінтах (`[Authorize]` , політики).
- **Небезпечна десеріалізація**, `BinaryFormatter` .
- **Слабка криптографія:** MD5/SHA1 для паролів → ASP.NET Core Identity, Argon2/PBKDF2.
- **Секрети в коді** та логах.
- **XSS / CSRF** у веб-застосунках.
- **Вразливі залежності:** `dotnet list package --vulnerable` , Dependabot.

Інструменти: GitHub Advanced Security (CodeQL, secret scanning), Roslyn security analyzers, SonarQube. Автоматизовані перевірки в CI — обов'язкові, бо ні людина, ні AI не бачать усього.

14.3. Приватність і конфіденційність

Питання	Що робити
Що відбувається з моїм кодом?	Читайте політику даних продукту й ваш тип плану (індивідуальний / бізнес / ентерпрайз); у бізнес-планах зазвичай діють жорсткіші гарантії щодо навчання на ваших даних
Персональні дані (PII)	Не вставляйте реальні дані клієнтів; анонімізуйте
Секрети	Не додавайте в чат, не тримайте в репозиторії
Політика компанії	Перед використанням з робочим кодом уточніть дозволені інструменти
Виключення файлів	У бізнес-планах Copilot можна налаштувати content exclusion для певних шляхів

14.4. Ліцензії та авторське право

- Модель може відтворити фрагменти з відкритого коду. У Copilot є фільтр збігів із публічним кодом (налаштування) — увімкніть, якщо вимагає політика.
- Для комерційних проєктів дотримуйтесь політики компанії щодо ліцензій залежностей і згенерованого коду.
- Відповідальність за код, який ви коммітите, — на вас.

14.5. Відповідальне використання

- Не делегуйте AI рішення, які потребують людського судження (безпека, фінанси, права людей).
- Ведіть журнал: які частини проєкту в основному згенеровані — корисно для підтримки.
- Продовжуйте вчитися: не дайте AI атрофувати базові навички. Пишіть складні частини самі, а AI просіть пояснювати й критикувати.

> **Завдання 14.** Запустіть `dotnet list package --vulnerable` у вашому проєкті; підключіть Dependabot або CodeQL у GitHub-репозиторії. Попросіть AI зробити security-рев'ю одного ендпоінту та перевірте вручну кожне зауваження.

Розділ 15. Свої AI-застосунки на .NET

15.1. Навіщо

До цього ми користувалися AI як інструментом. Тепер вбудуємо його у **власний продукт**: чат-бот підтримки, підсумовування документів, класифікація заявок, пошук за змістом.

15.2. Стек .NET для AI

Бібліотека	Призначення
Microsoft.Extensions.AI	Єдині абстракції (`IChatClient` , `IEmbeddingGenerator`) над різними провайдерами — як ILogger для AI
Azure OpenAI / OpenAI SDK	Доступ до GPT-моделей (хмара)
GitHub Models	Безкоштовне/дешеве експериментування з моделями через токен GitHub (для прототипів)
Ollama / локальні моделі	Запуск моделей на своєму ПК; без передачі даних назовні
Semantic Kernel	Оркестрація: плагіни (функції), планувальники, пам'ять
Microsoft Agent Framework	Побудова агентів і багатоагентних систем на .NET (перевіряйте актуальний стан продукту)
ModelContextProtocol SDK	Свої MCP-сервери/клієнти (розділ 10)
Vector stores (Azure AI Search, Qdrant, pgvector, SQLite-vec)	Сховище embeddings для RAG

Продукти й пакети швидко еволюціонують; назви й API звіряйте з документацією на Microsoft Learn.

15.3. Перший чат-запит

```
// dotnet add package Microsoft.Extensions.AI
// + пакет провайдера (напр. Microsoft.Extensions.AI.OpenAI)
IChatClient client = /* створений для вашого провайдера, ключ — з User Secrets */;

var response = await client.GetResponseAsync(
    "Поясни різницю між Task та ValueTask. Коротко, з прикладом.");
Console.WriteLine(response.Text);
```

Перевага абстракції: можна підмінити провайдера (Azure OpenAI ↔ Ollama) без переписування коду, а також додати middleware: логування, кешування, телеметрію.

15.4. Системний промпт і розмова

```
List<ChatMessage> history =
[
    new(ChatRole.System, "Ти асистент служби підтримки. Відповідай коротко, українською.
Якщо не знаєш — так і скажи."),
    new(ChatRole.User, "Як змінити пароль?")
];
var reply = await client.GetResponseAsync(history);
history.AddMessages(reply);   // зберегти для наступного кроку
```

API не має пам'яті: **ви** щоразу надсилаєте історію. Слідкуйте за довжиною (токени = гроші).

15.5. Function calling у вашому коді

Ви описуєте C#-методи як інструменти, модель вирішує, коли їх викликати:

```
[Description("Повертає статус замовлення за номером")]
static string GetOrderStatus(int orderId) => /* запит до БД */ "Shipped";

var options = new ChatOptions { Tools = [AIFunctionFactory.Create(GetOrderStatus)] };
var chat = new ChatClientBuilder(client).UseFunctionInvocation().Build();
var r = await chat.GetResponseAsync("Де моє замовлення 42?", options);
```

Модель лише *просить* виклик; виконує ваш код — тож перевіряйте аргументи й права доступу.

15.6. RAG: відповіді за вашими документами

1. **Підготовка:** розбийте документи на шматки (chunking), для кожного отримайте embedding, збережіть у векторному сховищі.
2. **Запит:** embedding питання → пошук найближчих шматків.
3. **Генерація:** додайте знайдені шматки в промпт: «Відповідай лише за наведеним контекстом; якщо відповіді немає — скажи про це».
4. **Цитування:** повертайте джерела, щоб користувач міг перевірити.

15.7. Структурований вивід

Для класифікації/витягування даних просіть JSON за схемою й десеріалізуйте у record. Бібліотеки дозволяють передати тип C# як цільову схему відповіді; завжди валідуйте результат.

15.8. Продакшн-питання

Тема	Що враховувати
Вартість	Лічіть токени; кешуйте; дешевша модель для простого
Затримки	Streaming-відповіді (`GetStreamingResponseAsync`)
Надійність	Retry/timeout (Polly), ліміти запитів, fallback
Безпека	Prompt injection, фільтрація вводу/виводу, мінімум прав у tools
Секрети	Key Vault / Managed Identity; ніколи в коді
Оцінювання (evals)	Набір тестових питань і автоперевірка якості при зміні промпта/моделі
Телеметрія	OpenTelemetry для викликів моделі
Конфіденційність	Що саме відправляється провайдеру; угоди про обробку даних

> **Фінальний проєкт.** Створіть консольний або Web API-застосунок «Помічник TodoApi»: користувач пише природною мовою («покажи невиконані задачі»), модель через function calling викликає ваші методи. Додайте системний промпт, логування й обробку помилок. Секрет ключа — у User Secrets. Запишіть README із архітектурою.

> **Далі — Том 6:** додатки — повний глосарій, шпаргалки, чеклісти, типові помилки.